REVISION CHEAT SHEET:

EXTENSIONS / C#

Extension Methods, Partial Types, and Anonymous Types in C#

Interview-ready notes · examples · C# · diagrams

Topic 25 of 40

This topic contains three C# features that solve very different problems:

EXAMPLE / FLOW

```
Extension methods
    →
Add convenient behavior to an existing type
without modifying that type
```

EXAMPLE / FLOW

```
Partial types
    →
Split one logical type across
multiple source declarations/files
```

EXAMPLE / FLOW

```
Anonymous types
    →
Create a temporary compiler-generated
read-only data shape without naming a class
```

These features often appear together in modern .NET applications because LINQ relies heavily on extension methods, source generators frequently rely on partial types/members, and LINQ projections frequently produce anonymous types.

C# 14 significantly expands extension methods into broader **extension members**, including extension properties and static extensions. Modern partial-member support has also expanded: C# 13 added partial properties/indexers, and C# 14 added partial instance constructors and partial events.

Part 1 — Extension Methods

1. What Is an Extension Method?

An extension method lets you call a static method using instance-method syntax.

Suppose .NET provides:

C#

```
string name = "annual campaign";
```

You don't own the `string` class.

You therefore can't simply edit `System.String` and add:

C#

```
ToTitleCase()
```

But an extension method can make this syntax possible:

EXTENSIONS / C#

```
C#
string result = name.ToTitleCase();
```

even though `ToTitleCase` isn't physically declared inside `System.String`.

Extension members let you associate new behavior with an existing type without modifying, recompiling, or deriving from that type. They still obey the original type's accessibility rules and don't gain privileged access to its private internals.

2. Classic Extension Method Syntax

The traditional syntax is:

```
C#
public static class GiftExtensions
{
    public static bool IsMajorGift(
        this Gift gift,
        decimal threshold)
    {
        ArgumentNullException.ThrowIfNull(gift);

        return gift.TotalAmount >= threshold;
    }
}
```

The important part is:

```
C#
this Gift gift
```

The first parameter:

EXAMPLE / FLOW

```
Gift
```

is the type being extended.

The keyword:

EXAMPLE / FLOW

```
this
```

marks that parameter as the extension receiver.

Calling It

Instead of:

```
C#
bool result = GiftExtensions.IsMajorGift(gift, 10_000m);
```

you can write:

```
C#
bool result = gift.IsMajorGift(10_000m);
```

This looks like an instance method.

But the implementation is still a static method.

Mental Model

Source code:

```C#
gift.IsMajorGift(10_000m);
```

Think conceptually:

```C#
GiftExtensions.IsMajorGift(
    gift,
    10_000m);
```

Extension syntax mainly provides:

EXAMPLE / FLOW

```
Convenient member-style syntax
+
discoverability through IntelliSense
+
composable APIs
```

3. Extension Methods Are Static Methods

This is an important interview point.

An extension method does **not** modify the runtime layout of the extended class.

This:

```C#
public static bool IsMajorGift(
    this Gift gift,
    decimal threshold)
```

still belongs to:

```C#
GiftExtensions
```

not:

```C#
Gift
```

The compiler simply permits:

```C#
gift.IsMajorGift(...)
```

syntax.

4. Extension Method Container Rules

Traditional extension methods must be defined inside a suitable static class.

Typical form:

```C#
namespace Fundraising.Application.Extensions;

public static class GiftExtensions
{
    public static bool IsMajorGift(
        this Gift gift,
        decimal threshold)
    {
        return gift.TotalAmount >= threshold;
    }
}
```

For both classic extension methods and C# 14 extension blocks, the extension container is a top-level, non-generic static class.

5. Importing the Namespace

Suppose the extension lives in:

```C#
namespace Fundraising.Application.Extensions;
```

Consumer code needs that namespace in scope:

```C#
using Fundraising.Application.Extensions;
```

Then:

```C#
bool major = gift.IsMajorGift(10_000m);
```

becomes available.

Extension members are brought into consideration through namespace/import scope rather than being physically injected into the target class.

6. Instance Members Beat Extension Methods

Suppose `Gift` contains:

```C#
public bool IsMajorGift(
    decimal threshold)
{
    return true;
}
```

and you also define:

```C#
public static bool IsMajorGift(
    this Gift gift,
    decimal threshold)
{
    return false;
}
```

Then:

```C#
gift.IsMajorGift(
    1000m);
```

calls the actual instance method.

Why?

Because real members declared on the type have higher priority than extension members.

Extension members cannot override an existing class/interface member with the same applicable signature.

Resolution Flow

```EXAMPLE / FLOW
gift.SomeMethod(...)
        ↓
Does Gift itself have
an applicable member?
        ↓
Yes
    →
Use real member
```

If not:

```EXAMPLE / FLOW
Search applicable
extension members
in scope
```

7. Extension Methods Are Resolved at Compile Time

Extension methods don't provide runtime polymorphism in the same way as:

```C#
virtual
```

and:

```C#
override
```

do.

Suppose:

```C#
public class Payment
{
}

public class GiftPayment : Payment
{
}
```

Extensions:

```C#
public static class PaymentExtensions
{
    public static string Describe(
        this Payment payment)
        =>
        "Payment";

    public static string Describe(
        this GiftPayment payment)
        =>
        "Gift Payment";
}
```

Then:

```C#
GiftPayment actual = new();

Payment reference = actual;

Console.WriteLine(
    reference.Describe());
```

The extension method is selected from the compile-time type of the receiver:

EXAMPLE / FLOW

```
Payment
```

not dynamically from:

EXAMPLE / FLOW

```
GiftPayment
```

Extension binding is compile-time member lookup, not virtual dispatch.

8. Extension Methods Do Not Break Encapsulation

Suppose:

```C#
public sealed class Gift
{
    private decimal _internalAmount;
}
```

An extension cannot do this:

```C#
public static decimal GetInternalAmount(
    this Gift gift)
{
    // return gift._internalAmount;
}
```

because:

EXAMPLE / FLOW

```
_internalAmount
```

is private to `Gift`.

The extension has only the same access that ordinary external code would have.

9. Null Receivers

This can surprise developers.

Suppose:

```C#
Gift? gift = null;
```

and:

```C#
public static bool IsMissing(
    this Gift? gift)
{
    return gift is null;
}
```

This can be called:

```C#
bool missing = gift.IsMissing();
```

Why?

Because an extension method is fundamentally a static call.

The null receiver is passed as the first argument.

The runtime does not automatically throw just because extension syntax was used; the extension itself decides how to handle a null receiver.

Important Design Rule

If null isn't valid:

```C#
public static decimal GetAllocatedAmount(
    this Gift gift)
{
    ArgumentNullException.ThrowIfNull(
        gift);

    return gift.Allocations.Sum(
        allocation =>
```

```
            allocation.Amount);
}
```

Make the contract explicit.

10. Generic Extension Methods

Extension methods work particularly well with generics.

Example:

```C#
public static class EnumerableExtensions
{
    public static bool HasAny<T>(
        this IEnumerable<T> source)
    {
        ArgumentNullException.ThrowIfNull(
            source);

        return source.Any();
    }
}
```

Usage:

```C#
bool hasGifts = gifts.HasAny();

bool hasCampaigns = campaigns.HasAny();
```

Same extension.

Different element types.

Generic Receiver with Constraint

```C#
public static class TenantExtensions
{
    public static bool BelongsToTenant<T>(
        this T entity,
        long tenantId)
        where T : ITenantScoped
    {
        ArgumentNullException.ThrowIfNull(
            entity);

        return entity.TenantId
            == tenantId;
    }
}
```

Usage:

```C#
gift.BelongsToTenant(
    tenantId);
```

```
campaign.BelongsToTenant(
    tenantId);
```

EXTENSIONS / C#

11. Extension Methods on Interfaces

This is extremely common.

LINQ itself works this way.

For example:

C#
```
gifts.Where(...)
```

`Where` isn't physically defined on:

C#
```
List<Gift>
```

It is exposed through extension methods over abstractions such as:

C#
```
IEnumerable<T>
```

That is one reason LINQ works across:

EXAMPLE / FLOW
```
Arrays

Lists

HashSets

Custom IEnumerable<T> sequences
```

Extension members are a foundational part of LINQ's fluent API design.

Fundraising Query Extension

A realistic reusable query extension:

C#
```
public static class GiftQueryExtensions
{
    public static IQueryable<Gift>
        ForTenant(
            this IQueryable<Gift> query,
            long tenantId)
    {
        ArgumentNullException.ThrowIfNull(
            query);

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        return query.Where(
            gift =>
                gift.TenantId
```

```
                == tenantId);
    }
}
```

EXTENSIONS / C#

Usage:

```C#
IQueryable<Gift> query = dbContext.Gifts.ForTenant(tenantId);
```

Then:

```C#
query = query.Where(gift => gift.Status == GiftStatus.Completed);
```

Security Warning

A helper such as:

```C#
ForTenant(...)
```

can improve consistency.

It does **not** become a security boundary merely because it is an extension method.

You still need a design that guarantees:

EXAMPLE / FLOW

```
Correct tenant identity

Authorization

Correct call paths

Correct database constraints
```

12. C# 14 Extension Blocks

C# 14 added a second syntax for defining extensions.

Instead of:

```C#
public static bool IsMajorGift(
    this Gift gift,
    decimal threshold)
```

you can write an extension block:

```C#
public static class GiftExtensions
{
    extension(Gift gift)
    {
        public bool IsMajorGift(
            decimal threshold)
        {
            return gift.TotalAmount
                >= threshold;
        }
```

```
    }
}
```

EXTENSIONS / C#

Call site remains:

```
C#
gift.IsMajorGift(
    10_000m);
```

Classic extension methods continue to work. Microsoft states that the old and new extension-method forms are source/binary compatible and generate equivalent callable behavior.

Why Add Extension Blocks?

The old syntax allowed:

```
EXAMPLE / FLOW
Extension methods
```

C# 14 extension blocks support broader:

```
EXAMPLE / FLOW
Extension members
```

including:

```
EXAMPLE / FLOW
Methods

Properties

Indexers

Static extensions

Operators
```

depending on the member declaration.

13. Extension Properties

Before C# 14, if you wanted:

```
EXAMPLE / FLOW
Gift → IsFullyAllocated
```

you needed a method:

```
C#
gift.IsFullyAllocated();
```

C# 14 can express it as an extension property:

```
C#
public static class GiftExtensions
{
    extension(Gift gift)
    {
```

```
        public bool IsFullyAllocated
            =>
            gift.Allocations.Sum(
                allocation =>
                    allocation.Amount)
            == gift.TotalAmount;
    }
}
```

EXTENSIONS / C#

Usage:

```C#
if (gift.IsFullyAllocated)
{
}
```

Extension Property Does Not Add Storage

An extension property doesn't physically add an instance field to:

```C#
Gift
```

It computes its value from accessible state.

Think:

```EXAMPLE / FLOW
Looks like property
    ↓
Implemented externally
    ↓
No new Gift instance storage
```

Design Caution

If:

```EXAMPLE / FLOW
IsFullyAllocated
```

is a fundamental Gift business invariant, it may belong directly inside:

```C#
Gift
```

rather than an extension.

Extension members should not become a way of moving important domain behavior out of the domain object simply for stylistic reasons.

14. Static Extension Members in C# 14

C# 14 also allows members that appear to extend the **type itself**.

Conceptually:

EXAMPLE / FLOW

```
Instance extension
    →
gift.SomeMember
```

EXAMPLE / FLOW

```
Static extension
    →
SomeType.SomeMember
```

Static extension members can include methods/properties, and C# 14 also supports operator extension scenarios.

These features are useful primarily for:

EXAMPLE / FLOW

```
Library design

Numeric abstractions

Factory-like APIs

Operator-centric types
```

They are less common in ordinary ASP.NET Core service code.

15. When Should You Use Extension Methods?

Good uses include:

EXAMPLE / FLOW

```
LINQ-style transformations

Formatting helpers

Reusable query composition

Conversion helpers

Framework integration helpers

Utilities over types you don't own
```

Example:

C#

```
public static string ToDisplayAmount(
    this decimal amount)
{
    return amount.ToString(
        "C");
}
```

When Should You Avoid Them?

Avoid extension methods when:

EXAMPLE / FLOW

```
Behavior is a core responsibility
of the class you own

The extension hides expensive I/O

The extension performs surprising side effects

The method needs substantial mutable state

An interface/service abstraction is clearer
```

EXTENSIONS / C#

Bad Example

C#

```
public static async Task SaveGiftAsync(
    this Gift gift)
{
    // Hidden database access.
}
```

Call site:

C#

```
await gift.SaveGiftAsync();
```

looks like an innocent domain operation.

But it may secretly:

EXAMPLE / FLOW

```
Open database connection

Execute SQL

Commit transaction
```

That can make architecture and testing harder to understand.

Better

C#

```
await giftRepository.SaveAsync(
    gift,
    cancellationToken);
```

The dependency and side effect are explicit.

Part 2 — Partial Types

16. What Is a Partial Type?

A partial type lets one logical type be split across multiple declarations.

File:

EXAMPLE / FLOW

GiftProcessor.Core.cs

EXTENSIONS / C#

C#

```
public partial class GiftProcessor
{
    public void Process(
        Gift gift)
    {
    }
}
```

Another file:

EXAMPLE / FLOW

GiftProcessor.Validation.cs

C#

```
public partial class GiftProcessor
{
    private bool Validate(
        Gift gift)
    {
        return gift.TotalAmount > 0m;
    }
}
```

At compile time they become:

EXAMPLE / FLOW

One GiftProcessor type

not two classes.

Partial declarations are combined by the compiler into one type. C# supports partial classes, structs, interfaces, and records, provided the declarations are available together during compilation.

Runtime Mental Model

Source files:

EXAMPLE / FLOW

```
GiftProcessor.Core.cs
        +
GiftProcessor.Validation.cs
        +
GiftProcessor.Generated.cs
```

Compile:

EXAMPLE / FLOW

```
        ↓
```

Runtime type:

EXAMPLE / FLOW

GiftProcessor

There is no concept at runtime of:

EXAMPLE / FLOW

```
"the validation half"
```

versus:

EXAMPLE / FLOW

```
"the generated half"
```

It is one compiled type.

17. Every Declaration Uses `partial`

This is incorrect:

C#

```
public partial class GiftProcessor
{
}
```

plus:

C#

```
public class GiftProcessor
{
}
```

All declarations participating in the split type must declare:

C#

```
partial
```

18. Partial-Type Rules

Important rules include:

- all parts represent the same type;
- all parts must be available to the same compilation;
- they must agree on accessibility;
- generic type parameters must match;
- different parts cannot specify conflicting base classes;
- interfaces listed across different parts are combined;
- members from one part are visible to the other parts.

Partial definitions intended to form the same type must belong to the same assembly/module and must be compiled together.

Interface Combination Example

Part one:

C#

```
public partial class GiftProcessor :
    IGiftProcessor
```

```
{
}
```

EXTENSIONS / C#

Part two:

```C#
public partial class GiftProcessor :
    IDisposable
{
}
```

The final type implements both:

```EXAMPLE / FLOW
IGiftProcessor

IDisposable
```

Partial type interface lists are combined.

Base Class Conflict

This is invalid conceptually:

```C#
partial class GiftProcessor :
    BaseA
{
}
```

and:

```C#
partial class GiftProcessor :
    BaseB
{
}
```

A class can have only one base class.

Partial declarations that specify a base class must agree.

19. Primary Use Case — Generated Code

Partial types are particularly valuable when one part is generated.

Example:

```EXAMPLE / FLOW
Human-written file
    +
Source-generated file
```

The generator can safely regenerate its file without overwriting your hand-written code.

This pattern is used by many tooling and source-generation scenarios. Microsoft specifically identifies generated code and source generators as major use cases for partial types and partial members.

Good File Layout

EXTENSIONS / C#

EXAMPLE / FLOW

```
GiftImportProcessor.cs
    →
Human-maintained logic

GiftImportProcessor.Generated.cs
    →
Generated implementation
```

The rule should usually be:

> **Do not manually modify generated files.**

Instead, customize the human-maintained partial declaration.

20. Partial Does Not Mean "Break Giant Class into Files and Keep Growing"

You could write:

EXAMPLE / FLOW

```
GiftService.Create.cs

GiftService.Update.cs

GiftService.Delete.cs

GiftService.Validation.cs

GiftService.Email.cs

GiftService.Reporting.cs

GiftService.Integration.cs
```

all as one gigantic:

C#

```
partial class GiftService
```

That may reduce file size.

It does **not** reduce the responsibilities of the type.

You still have:

EXAMPLE / FLOW

```
One class

One dependency graph

One responsibility problem
```

If the class violates SRP, splitting it into eight files doesn't fix the design.

Use Partial Primarily When

EXAMPLE / FLOW

```
Generated/manual code must coexist

Tooling supplies part of a type

A framework requires partial declarations

Source generators implement members

A type has a legitimate source-level split
```

Do not use it merely to hide an oversized design.

21. Partial Methods

Partial members allow one declaration to define a member contract and another declaration to provide the implementation.

Classic partial method:

C#

```
public partial class GiftProcessor
{
    partial void OnGiftProcessed(
        Gift gift);

    public void Process(
        Gift gift)
    {
        // Processing...

        OnGiftProcessed(
            gift);
    }
}
```

Another file:

C#

```
public partial class GiftProcessor
{
    partial void OnGiftProcessed(
        Gift gift)
    {
        Console.WriteLine(
            $"Processed {gift.GiftId}");
    }
}
```

Defining vs Implementing Declaration

First:

C#

```
partial void OnGiftProcessed(
    Gift gift);
```

is the:

```
EXAMPLE / FLOW
Defining declaration
```

Second:

```
C#
partial void OnGiftProcessed(
    Gift gift)
{
}
```

is the:

```
EXAMPLE / FLOW
Implementing declaration
```

The signatures must match.

22. Optional Classic Partial Methods

A special form of partial method doesn't require an implementation.

For that optional form, the method must satisfy restrictions including:

```
EXAMPLE / FLOW
No explicit accessibility modifier

void return

No out parameters

No virtual / override / sealed /
new / extern
```

If no implementation exists, the compiler removes the method declaration and calls to it.

Example

```
C#
partial class GiftImporter
{
    partial void OnGiftParsed(
        Gift gift);

    private void Parse(
        Gift gift)
    {
        OnGiftParsed(
            gift);
    }
}
```

If no implementation exists, the call contributes no runtime behavior.

This historically made partial methods useful as generated-code hooks.

23. Modern Partial Methods

EXTENSIONS / C#

Modern C# also allows more capable partial methods.

Example:

C#

```
public partial class GiftNormalizer
{
    public partial string NormalizeReference(
        string reference);
}
```

Because it is:

EXAMPLE / FLOW

```
public
```

and returns:

EXAMPLE / FLOW

```
string
```

an implementation is required.

Another declaration:

C#

```
public partial class GiftNormalizer
{
    public partial string NormalizeReference(
        string reference)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(
            reference);

        return reference
            .Trim()
            .ToUpperInvariant();
    }
}
```

Partial members that don't meet the optional-partial-method restrictions must have an implementation.

24. Partial Members and Source Generators

This pattern is extremely useful with source generators.

Human source:

C#

```
public static partial class
    GiftReferencePatterns
{
    [GeneratedRegex(
        @"^GFT-\d+$")]
    public static partial Regex
        GiftReference();
}
```

Your code declares:

EXAMPLE / FLOW

```
What member should exist
```

EXTENSIONS / C#

A source generator creates:

EXAMPLE / FLOW

```
Its implementation
```

during compilation.

Partial-member support exists specifically to enable tooling/source generators to provide implementations while your source keeps the public declaration.

25. Partial Properties — C# 13+

C# 13 expanded partial members to include:

EXAMPLE / FLOW

```
Properties

Indexers
```

Example defining declaration:

C#

```
public partial class GiftSnapshot
{
    public partial decimal Amount
    {
        get;
        set;
    }
}
```

Implementation:

C#

```
public partial class GiftSnapshot
{
    private decimal _amount;

    public partial decimal Amount
    {
        get =>
            _amount;

        set
        {
            if (value < 0m)
            {
                throw new ArgumentOutOfRangeException(
                    nameof(value));
            }

            _amount = value;
        }
    }
}
```

Partial properties require matching defining and implementing declarations; the implementing declaration cannot simply be another auto-property.

Why Partial Properties?

Their primary value is tooling.

A generator can declare or discover:

```
C#
public partial string Name
{
    get;
    set;
}
```

while another part provides specialized implementation.

This should not be your default way to write ordinary properties.

26. C# 14 Partial Constructors and Events

C# 14 expands partial members again to include:

```
EXAMPLE / FLOW
Instance constructors

Events
```

This makes code-generation scenarios more flexible, because generated and user-written parts can cooperate around more kinds of members.

These are advanced features.

For ordinary hand-written business classes:

```
EXAMPLE / FLOW
normal constructor
normal event
```

is usually clearer.

Partial-Member Timeline

```
EXAMPLE / FLOW
Earlier C#
    →
Partial methods
```

```
EXAMPLE / FLOW
C# 13
    →
Partial properties
Partial indexers
```

```
EXAMPLE / FLOW
C# 14
    →
```

```
Partial instance constructors
Partial events
```


27. Attributes on Partial Types

Suppose one declaration contains:

```C#
[Serializable]
public partial class GiftSnapshot
{
}
```

and another contains another supported attribute.

Attributes declared across partial type parts are combined by the compiler.

Documentation comments can also come from multiple partial declarations according to the language rules.

28. Partial Type and Encapsulation

Because all parts become one type, this:

```C#
public partial class GiftProcessor
{
    private int _processedCount;
}
```

can be accessed by another part:

```C#
public partial class GiftProcessor
{
    private void Increment()
    {
        _processedCount++;
    }
}
```

The second file isn't an external collaborator.

It is literally another declaration of the same class.

Part 3 — Anonymous Types

29. What Is an Anonymous Type?

An anonymous type lets you create a small read-only object without declaring a named class first.

Example:

```C#
var giftSummary =
    new
    {
        GiftId =
            gift.GiftId,
```

```
    Amount = gift.TotalAmount };
```

The compiler generates the actual type.

You don't name it yourself.

Anonymous types are compiler-generated reference types containing inferred read-only properties and are particularly common in LINQ projections.

Without Anonymous Type

You might need:

```C#
public sealed class GiftSummary
{
    public long GiftId
    {
        get;
        init;
    }

    public decimal Amount
    {
        get;
        init;
    }
}
```

Then:

```C#
GiftSummary summary =
    new()
    {
        GiftId =
            gift.GiftId,

        Amount = gift.TotalAmount };
```

If that shape is required only temporarily inside one method, creating a permanent named class may be unnecessary.

Anonymous Version

```C#
var summary =
    new
    {
        gift.GiftId,
        Amount = gift.TotalAmount };
```

30. Why var Is Required

You cannot write:

```C#
SomeAnonymousType summary = new { gift.GiftId, gift.TotalAmount };
```

because your source code doesn't know the compiler-generated type name.

Therefore:

EXTENSIONS / C#

```
C#
var summary = new { gift.GiftId, gift.TotalAmount };
```

is the normal syntax.

Because the generated type can't be named in source, anonymous-type local variables are declared using `var`.

Important

`var` does **not** mean:

```
EXAMPLE / FLOW
dynamic
```

The compiler still knows the anonymous type exactly.

Therefore:

```
C#
Console.WriteLine(
    summary.GiftId);
```

is compile-time checked.

This would fail at compile time:

```
C#
// summary.UnknownProperty
```

31. Inferred Property Names

This:

```
C#
var summary = new { gift.GiftId, gift.TotalAmount };
```

creates properties named approximately:

```
EXAMPLE / FLOW
GiftId

TotalAmount
```

because those member-access names can be inferred.

Microsoft calls this property-name inference/projection initialization.

Explicit Property Name

```
C#
var summary =
    new
    {
        Id =
```

```
        gift.GiftId,
    Amount = gift.TotalAmount };
```

EXTENSIONS / C#

Now property names are:

EXAMPLE / FLOW

```
Id

Amount
```

32. Anonymous Properties Are Read-Only

This:

C#

```
var gift = new { GiftId = 1001L, Amount = 500m };
```

does not allow:

C#

```
// gift.Amount = 1000m;
```

Anonymous-type properties are public and read-only.

33. Anonymous Types Are Reference Types

Current C# compilers generate anonymous types as internal sealed classes deriving from `object`.

Conceptually:

EXAMPLE / FLOW

```
Anonymous object
    ↓
Compiler-generated sealed class
    ↓
Reference type
```

Do not confuse them with tuples, which are normally:

EXAMPLE / FLOW

```
ValueTuple
    →
value types
```

34. Anonymous-Type Equality

This is a very important difference from a normal plain class.

Consider:

C#

```
var first = new { Name = "Annual", Amount = 100m };

var second = new { Name = "Annual", Amount = 100m };
```

Then:

```C#
bool equal = first.Equals(second);
```

returns:

```EXAMPLE / FLOW
true
```

when the anonymous instances have the same generated anonymous type and equal property values.

The compiler supplies value-based `Equals` and `GetHashCode` behavior for anonymous types.

What Determines the Generated Anonymous Type?

Within the same compiled program/assembly, anonymous initializers correspond to the same generated anonymous type when they have matching:

```EXAMPLE / FLOW
Property names

Property compile-time types

Property order
```

Same Shape

```C#
var first = new { Id = 1L, Name = "A" };

var second = new { Id = 2L, Name = "B" };
```

Same property:

```EXAMPLE / FLOW
names

types

order
```

so they have the same anonymous type.

Different Order

```C#
var first = new { Id = 1L, Name = "A" };

var second = new { Name = "B", Id = 2L };
```

The property order differs.

Therefore the generated anonymous type shape differs.

35. Anonymous Types and LINQ

This is their most common real-world use.

```
C#
var query =
    gifts.Select(
        gift =>
            new
            {
                gift.GiftId,
                gift.TotalAmount
            });
```

Each element now has a temporary shape:

```
EXAMPLE / FLOW
GiftId

TotalAmount
```

without defining:

```
C#
GiftProjection
```

Anonymous types are especially useful for intermediate LINQ projections.

Grouping Example

```
C#
var campaignTotals =
    gifts
        .GroupBy(
            gift =>
                gift.CampaignId)
        .Select(
            group =>
                new
                {
                    CampaignId =
                        group.Key,

                    GiftCount =
                        group.Count(),

                    TotalAmount = group.Sum(gift => gift.TotalAmount) });
```

This is ideal when:

```
EXAMPLE / FLOW
Result exists only inside
the current implementation
```

36. Anonymous Types with EF Core

Anonymous types are useful inside EF Core projections:

```
C#
var data =
    await dbContext.Gifts
        .Where(
            gift =>
```

```
                gift.TenantId
                ==
                tenantId)
        .Select(
            gift =>
                new
                {
                    gift.GiftId,
                    gift.TotalAmount
                })
        .ToListAsync(
            cancellationToken);
```

EXTENSIONS / C#

This lets the query provider project only the requested shape.

But Avoid Anonymous Public API Contracts

Controller:

```C#
return Ok(
    new
    {
        gift.GiftId,
        gift.TotalAmount
    });
```

can technically serialize.

But for stable production API contracts, a named DTO is usually clearer:

```C#
public sealed record GiftResponse(
    long GiftId,
    decimal TotalAmount);
```

Why?

Because a named contract provides:

EXAMPLE / FLOW

```
Discoverable type

Reusable documentation

Compile-time API boundary

Validation/annotations where needed

Stable evolution
```

37. Anonymous Types Cannot Normally Be Method Contract Types

You can't declare:

```C#
public ??? GetSummary()
```

where `???` is the anonymous type.

Likewise you can't name it as a normal:

EXTENSIONS / C#

EXAMPLE / FLOW

```
Field type

Parameter type

Return type
```

because source code has no accessible type name for it.

Don't Escape with `object`

You could technically write:

C#

```
public object GetSummary()
{
    return new
    {
        Id = 1,
        Amount = 100m
    };
}
```

but now the caller sees only:

C#

```
object
```

and loses normal static access to the anonymous properties.

This usually defeats the benefit.

Don't Escape with `dynamic` Without a Good Reason

C#

```
public dynamic GetSummary()
```

moves validation from compile time toward runtime.

For application boundaries, prefer a named:

EXAMPLE / FLOW

```
Record

Class

DTO
```

38. Anonymous `with` Expressions

Modern C# supports nondestructive copying of anonymous objects.

Example:

```C#
var original = new { GiftId = 1001L, Amount = 500m };

var updated = original with { Amount = 750m };
```

`original` remains unchanged.

`updated` gets the modified value.

Anonymous types support `with` expressions while their properties remain read-only.

39. Nested Anonymous Types

You can nest them:

```C#
var result =
    new
    {
        gift.GiftId,

        Constituent =
            new
            {
                constituent.ConstituentId,
                constituent.DisplayName
            },

        Amount = gift.TotalAmount };
```

Useful for:

```EXAMPLE / FLOW
Temporary hierarchy

LINQ analysis

Debugging

Internal projections
```

40. Anonymous Type vs Tuple

Anonymous:

```C#
var result = new { gift.GiftId, gift.TotalAmount };
```

Tuple:

```C#
var result = (gift.GiftId, gift.TotalAmount);
```

Both can represent temporary compound values.

Comparison

EXTENSIONS / C#

Feature	Anonymous Type	Tuple
Named type required	No	No
Reference/value type	Reference type	Usually `ValueTuple`, value type
Read-only properties	Yes	Tuple fields can be assigned
Value-based equality	Yes	Yes
Deconstruction	Not native tuple syntax	Yes
LINQ projections	Excellent	Good
Expression-tree scenarios	Supported	More restricted
Public API	Poor choice	Sometimes acceptable internally/publicly
Property-like names	Yes	Element names
`with` support	Yes	Not the same model

Microsoft's current guidance notes that tuples are often convenient for temporary data, while anonymous types remain particularly useful for query/expression-tree scenarios and reference-type semantics.

41. Anonymous Type vs Record

Anonymous:

C#
```
var response = new { gift.GiftId, gift.TotalAmount };
```

Named record:

C#
```
public sealed record GiftResponse(
    long GiftId,
    decimal TotalAmount);
```

Use anonymous types for:

EXAMPLE / FLOW
```
Short-lived internal shape
```

Use a record/DTO for:

EXAMPLE / FLOW
```
API boundary

Method contract

Reusable result

Documented business/application shape
```

How the Three Features Relate

These three topics solve fundamentally different problems:

EXAMPLE / FLOW

```
Extension method
    →
Add external behavior
```

EXTENSIONS / C#

EXAMPLE / FLOW

```
Partial type
    →
Split one named type's implementation
```

EXAMPLE / FLOW

```
Anonymous type
    →
Create temporary unnamed data shape
```

Do not mix their purposes.

Extension Method vs Partial Class

Suppose you own:

C#

```
Gift
```

and want to add core behavior.

Extension approach:

C#

```
gift.CompleteAllocation();
```

via external static helper.

Partial approach:

C#

```
partial class Gift
{
    public void CompleteAllocation()
    {
    }
}
```

if `Gift` itself is already partial.

But the real question is:

Should this behavior genuinely belong to Gift?

If yes and you own the type:

EXAMPLE / FLOW

```
Normal class member
```

is generally clearer than an extension.

Extension members are especially useful when you don't control the original type or when the functionality is genuinely auxiliary. Microsoft likewise recommends modifying/deriving the type w
EXTENSIONS / C#
using extension members when external extension is the appropriate design.

Performance Considerations

Extension Methods

A traditional extension call is fundamentally a static method call.

Don't assume:

EXAMPLE / FLOW
```
extension syntax
=
virtual dispatch
```

The performance concern is usually the work inside the method, not the extension syntax itself.

Hidden Expensive Work

Avoid extension methods that look trivial but perform:

EXAMPLE / FLOW
```
Database calls

Network calls

File I/O

Large allocations
```

Example:

C#
```
await gift.LoadAllHistoryAsync();
```

looks like a simple object operation.

If it secretly creates an entire database workflow, the API may be misleading.

Partial Types

Splitting a type across files does not inherently create:

EXAMPLE / FLOW
```
More runtime objects

Extra calls

Extra allocation
```

The compiler merges the declarations into one type.

The primary concerns are:

EXAMPLE / FLOW
```
Maintainability

Generated-code ownership
```

```
Design cohesion
```


Anonymous Types

Anonymous objects are still objects.

Example:

```C#
var result = new { gift.GiftId, gift.TotalAmount };
```

creates an instance when executed.

Don't assume:

```EXAMPLE / FLOW
Anonymous
=
free
```

But anonymous projections can improve database performance when they allow EF Core to select only needed columns.

Security Considerations

Extension Query Helpers

A helper such as:

```C#
query.ForTenant(
    tenantId)
```

is useful.

But developers may accidentally bypass it.

Do not base your entire authorization model on remembering to call one extension.

Anonymous API Responses

Be careful with:

```C#
return new
{
    constituent.FullName,
    constituent.Email,
    constituent.InternalNotes,
    constituent.SensitiveData
};
```

Anonymous types make ad-hoc projections very easy.

That doesn't mean every available field should be serialized.

Explicit DTOs often make security review easier.

Generated Partial Code

Generated files should be treated as generated artifacts.

Do not place:

```
EXAMPLE / FLOW
Secrets

Credentials

Environment-specific sensitive values
```

inside code-generation output merely because the source is partial.

Generated code still becomes normal compiled application code.

Common Mistake 1 — Thinking Extension Method Modifies Original Class

It does not.

```
C#
gift.IsMajorGift()
```

may look like a `Gift` member.

But the implementation still belongs to a static extension container.

Common Mistake 2 — Trying to Override with Extension Method

Extension members cannot override actual instance members.

Real members win member lookup.

Common Mistake 3 — Expecting Runtime Polymorphism

Extension selection is compile-time based.

If runtime polymorphic behavior is required:

```
EXAMPLE / FLOW
virtual / override

interface dispatch

strategy
```

is usually more appropriate.

Common Mistake 4 — Forgetting the Namespace

You create:

```
C#
GiftExtensions
```

but:

C#

```
gift.IsMajorGift()
```

EXTENSIONS / C#

doesn't appear.

Often the missing piece is:

C#

```
using Fundraising.Application.Extensions;
```

Common Mistake 5 — Putting Core Domain Rules in Extensions

Bad design:

C#

```
gift.MakeValid();
```

implemented externally while `Gift` itself permits invalid state.

If it is a true invariant, it usually belongs in the domain object's own behavior.

Common Mistake 6 — Hiding I/O

Avoid:

C#

```
gift.Save();
```

as an extension that silently performs database operations.

Extension methods should not make expensive side effects surprising.

Common Mistake 7 — Partial Class as God-Class Organization

Splitting:

EXAMPLE / FLOW

```
GiftService.cs
```

into ten files doesn't make one huge service better designed.

Partial changes:

EXAMPLE / FLOW

```
source organization
```

not:

EXAMPLE / FLOW

```
object responsibility
```

Common Mistake 8 — Editing Generated Partial Files

The next build/source-generation step may overwrite your changes.

Put custom logic in the user-maintained partial declaration instead.

Common Mistake 9 — Mismatched Partial Declarations

These must agree on important type/member details.

Conflicting:

EXAMPLE / FLOW

```
Accessibility

Base classes

Generic parameters

Partial-member signatures
```

cause compile-time errors.

Common Mistake 10 — Assuming Every Partial Method Can Be Omitted

Only the restricted classic optional form may omit its implementation.

A:

C#

```
public partial string Normalize(...);
```

requires an implementation.

Common Mistake 11 — Implementing Partial Property as Auto-Property

This isn't the correct implementing declaration:

C#

```
public partial string Name
{
    get;
    set;
}
```

if that is meant to be the implementation.

That syntax represents the defining declaration.

The implementing declaration needs accessor bodies.

Common Mistake 12 — Anonymous Type as Domain Model

Avoid:

C#

```
var gift =
    new
    {
        Id = 1,
        Amount = 100m,
```

```
        Status = "Completed"
};
```

as a substitute for a real:

```
C#
Gift
```

domain model.

Anonymous types are data shapes, not a replacement for rich domain types.

Common Mistake 13 — Anonymous Type as Public API Contract

Prefer:

```
C#
GiftResponse
```

over ad-hoc anonymous JSON for stable production endpoints.

Common Mistake 14 — Thinking `var` Means Dynamic

```
C#
var item = new { Id = 1 };
```

is fully statically typed.

The type's name is anonymous to your source code.

Its structure is not unknown to the compiler.

Common Mistake 15 — Ignoring Anonymous Property Order

These shapes:

```
C#
new
{
    Id = 1,
    Name = "A"
}
```

and:

```
C#
new
{
    Name = "A",
    Id = 1
}
```

aren't the same anonymous type because property order contributes to the generated type identity.

Best Practices

Use extension methods when:

EXAMPLE / FLOW

```
You don't control the original type

Behavior is auxiliary

A fluent API improves readability

You are building reusable query/helper operations
```

EXTENSIONS / C#

Prefer normal members when:

EXAMPLE / FLOW

```
You own the type

Behavior is fundamental to its responsibility
```

Use extension methods with meaningful names and predictable behavior.

Keep extension namespaces intentional; importing a namespace can expose many extension members.

Remember that actual members win over extensions.

Don't expect virtual dispatch from extension methods.

Guard null receivers when null is invalid.

For C# 14, use extension blocks when extension properties or grouped extension members make the API clearer.

Do not adopt new extension-member syntax merely because it is new.

Use partial types primarily where:

EXAMPLE / FLOW

```
Generated code

Source generators

Tooling

Legitimate generated/manual separation
```

benefit from them.

Don't use partial types as an excuse for low cohesion.

Keep generated and hand-written files clearly separated.

Use partial members when tooling needs to declare/implement hooks.

Understand whether a partial member's implementation is optional or required.

Use anonymous types for:

EXAMPLE / FLOW

```
Temporary LINQ projections

Intermediate calculations

Local grouping/reporting shapes
```

Prefer records/classes/DTOs for:

EXAMPLE / FLOW

EXTENSIONS / C#

```
Public API contracts

Layer boundaries

Reusable results

Domain models
```

Practical Exercise 1 — Extension Method

Create:

C#

```
public static class GiftExtensions
{
    public static decimal AllocationTotal(
        this Gift gift)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        return gift.Allocations.Sum(
            allocation =>
                allocation.Amount);
    }
}
```

Usage:

C#

```
decimal allocated = gift.AllocationTotal();
```

Practical Exercise 2 — Generic Extension

C#

```
public static bool IsEmpty<T>(
    this IEnumerable<T> source)
{
    ArgumentNullException.ThrowIfNull(
        source);

    return !source.Any();
}
```

Usage:

C#

```
bool noGifts = gifts.IsEmpty();
```

Practical Exercise 3 — Query Extension

C#

```
public static IQueryable<Gift>
    Completed(
        this IQueryable<Gift> query)
{
```

```
    return query.Where(
        gift =>
            gift.Status
            == GiftStatus.Completed);
}
```

EXTENSIONS / C#

Usage:

```C#
var query = dbContext.Gifts.Completed();
```

Practical Exercise 4 — Instance Member Precedence

If `Gift` declares:

```C#
public bool IsValid()
```

and an extension also declares:

```C#
public static bool IsValid(
    this Gift gift)
```

which runs?

Answer:

EXAMPLE / FLOW

```
Gift.IsValid()
```

The real instance member has priority.

Practical Exercise 5 — Partial Class

File one:

```C#
public partial class GiftImporter
{
    public void Import()
    {
        Validate();
    }
}
```

File two:

```C#
public partial class GiftImporter
{
    private void Validate()
    {
    }
}
```

Question:

How many runtime types exist?

Answer:

EXTENSIONS / C#

EXAMPLE / FLOW

```
One GiftImporter type
```

Practical Exercise 6 — Optional Partial Method

C#

```
partial class GiftImporter
{
    partial void OnImported(
        long giftId);
}
```

If no implementation exists and it satisfies the optional restrictions:

EXAMPLE / FLOW

```
Compiler removes the partial hook/calls.
```

Practical Exercise 7 — Required Partial Method

C#

```
public partial class GiftFormatter
{
    public partial string Format(
        Gift gift);
}
```

Question:

Can its implementation be omitted?

Answer:

EXAMPLE / FLOW

```
No.
```

Its accessibility and return value mean an implementing declaration is required.

Practical Exercise 8 — Anonymous Projection

C#

```
var summaries =
    gifts.Select(
        gift =>
            new
            {
                gift.GiftId,
                gift.TotalAmount
            });
```

No named DTO is needed if the result stays within the local implementation.

Practical Exercise 9 — Anonymous Equality

```C#
var first = new { Id = 1, Name = "Gift" };

var second = new { Id = 1, Name = "Gift" };
```

Expected:

```C#
first.Equals(
    second)
```

is:

```EXAMPLE / FLOW
true
```

because the generated type and property values match.

Practical Exercise 10 — Order Matters

```C#
var first = new { Id = 1, Name = "Gift" };

var second = new { Name = "Gift", Id = 1 };
```

These aren't the same generated anonymous type shape.

Practical Exercise 11 — DTO Decision

Requirement:

```EXAMPLE / FLOW
Temporary calculation inside one method
```

Use:

```EXAMPLE / FLOW
Anonymous type
```

Requirement:

```EXAMPLE / FLOW
ASP.NET Core response contract
```

Prefer:

```EXAMPLE / FLOW
Named DTO / record
```

Practical Exercise 12 — Choose the Feature

Requirement:

EXAMPLE / FLOW

Add a helper to IEnumerable<Gift>

EXTENSIONS / C#

Choose:

EXAMPLE / FLOW

Extension method

Requirement:

EXAMPLE / FLOW

Source generator provides implementation
of part of a class

Choose:

EXAMPLE / FLOW

Partial type/member

Requirement:

EXAMPLE / FLOW

Temporary LINQ projection

Choose:

EXAMPLE / FLOW

Anonymous type

Exactly 25 Interview Questions

1. What is an extension method in C#?

An extension method is a static method that can be invoked using instance-member syntax on another type.

Classic syntax uses:

```C#
this
```

on the first parameter.

It doesn't physically modify the extended class.

2. What are the requirements for declaring a traditional extension method?

It is declared as a static method in an appropriate top-level, non-generic static container class, and the first parameter uses:

```C#
this
```

to specify the receiver type.

3. Is an extension method really an instance method?

No.

This:

```
C#
gift.IsMajorGift();
```

is instance-style syntax over a static extension implementation.

4. What happens if a class has an instance method with the same applicable signature as an extension method?

The instance member wins.

Extension members cannot override members declared by the original type.

5. Are extension methods polymorphic like virtual methods?

No.

Extension member binding is primarily compile-time member resolution.

Use:

```
C#
virtual / override
```

or interfaces when runtime dispatch is required.

6. Can extension methods access private members of the extended type?

No.

They receive no special encapsulation privileges.

They can only access members that would normally be accessible to code in their location.

7. Can an extension method be called on a null reference?

Yes, because the receiver becomes an argument to the underlying static call.

The extension itself must decide whether null is valid or should cause an exception.

8. What are C# 14 extension blocks?

C# 14 introduces:

```
C#
extension(...)
{
}
```

blocks inside extension container classes.

They provide another syntax for extension methods and also support broader extension-member forms.

9. What extension members can C# 14 define?

C# 14 extension blocks can support extension methods and properties and also advanced member forms such as indexers, static extensions, and operators according to the member rules.

10. When should extension methods be avoided?

EXTENSIONS / C#

Avoid them when they hide core domain behavior, surprising side effects, significant I/O, or when a normal member/service/interface gives the design clearer ownership.

11. What is a partial class?

A partial class is a class whose declaration is split across multiple source declarations.

The compiler combines all parts into one class.

12. What kinds of types can be partial?

Modern C# supports partial:

EXAMPLE / FLOW

```
Classes

Structs

Interfaces

Records
```

The declarations must participate in the same compilation.

13. Why are partial types useful?

Their most important uses are:

EXAMPLE / FLOW

```
Generated code

Source generators

Designer/tool-generated code

Separating generated and hand-written portions
```

14. Are two files containing a partial class two runtime classes?

No.

They compile into one type.

Members from every partial declaration belong to the same resulting class.

15. Can two parts of a partial class have different accessibility?

No.

Partial declarations must agree on accessibility and other required type-definition details.

16. Can different partial declarations specify different interfaces?

Yes.

Interface lists across partial declarations are combined into the resulting type, provided the declarations otherwise form a valid partial type.

17. What is a partial method?

EXTENSIONS / C#

A partial method/member has a defining declaration and potentially a separate implementing declaration.

The declarations must have compatible matching signatures.

18. Can a partial method omit its implementation?

Only the restricted classic optional form can.

It must, among other requirements, return `void`, have no explicit accessibility, contain no `out` parameter, and avoid certain modifiers.

If such an implementation is omitted, the compiler removes the method/calls.

19. Which partial-member features were added in C# 13 and C# 14?

C# 13 added:

EXAMPLE / FLOW

```
Partial properties

Partial indexers
```

C# 14 added:

EXAMPLE / FLOW

```
Partial instance constructors

Partial events
```

20. What is an anonymous type?

An anonymous type is a compiler-generated read-only reference type whose name isn't accessible directly in source code.

It is created using syntax such as:

C#

```
var value = new { Id = 1, Name = "Gift" };
```

21. Why do anonymous-type variables normally use var?

Because the compiler generates the type name and source code can't explicitly refer to that generated name.

`var` still preserves full static typing.

22. Are anonymous-type properties mutable?

No.

Their generated properties are read-only.

Modern anonymous types can use `with` expressions to create modified copies rather than mutating existing properties.

23. How does equality work for anonymous types?

Anonymous types use compiler-generated value-based `Equals` and `GetHashCode`.

Instances with the same generated anonymous type are equal when their corresponding property values are equal.

24. What determines whether two anonymous object initializers generate the same anonymous type?

Within the relevant compilation context, they must have corresponding properties with the same:

EXAMPLE / FLOW

```
Names

Compile-time types

Order
```

25. When should you use extension methods, partial types, and anonymous types in an ASP.NET Core application?

Use:

EXAMPLE / FLOW

```
Extension methods
    →
Reusable helper/query behavior

Partial types
    →
Generated/manual source cooperation

Anonymous types
    →
Temporary local/query projections
```

Prefer named services for major behavior and named DTOs/records for stable API/layer contracts.

Quick Revision

Feature	Purpose
Extension method	Add callable behavior externally
`this` receiver	Classic extension syntax
Extension block	C# 14 extension-member syntax
Extension property	C# 14 externally defined property-like member
Real member vs extension	Real member wins
Extension dispatch	Compile-time
Partial type	Split one type across declarations
Partial class	Multiple declarations → one class
Partial method	Definition + optional/required implementation
Partial property	C# 13+
Partial indexer	C# 13+
Partial constructor	C# 14+

Feature	Purpose
Partial event	C# 14+
Anonymous type	Temporary compiler-generated shape
Anonymous properties	Read-only
Anonymous equality	Value-based
`var`	Required to hold anonymous type locally
Anonymous + LINQ	Common projection pattern

EXTENSIONS / C#

Complete Fundraising Example

Suppose an endpoint needs to query Gifts.

We want:

EXAMPLE / FLOW

```
Reusable query composition

Generated validation support

Temporary reporting projection
```

Each of today's features can serve a different role.

Extension Method — Reusable Query

C#

```
public static class GiftQueryExtensions
{
    public static IQueryable<Gift>
        ForTenant(
            this IQueryable<Gift> query,
            long tenantId)
    {
        ArgumentNullException.ThrowIfNull(
            query);

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        return query.Where(
            gift =>
                gift.TenantId
                == tenantId);
    }

    public static IQueryable<Gift>
        Completed(
            this IQueryable<Gift> query)
    {
        ArgumentNullException.ThrowIfNull(
            query);
```

```
    return query.Where(
```